

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING

NAME: K.DEEPIKA

COURSE CODE: CSA43

REG.NO:192371042

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS:

Total Marks: 20

Question 1 (5 Marks)

Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.

Question 2 (5 Marks)

Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in

validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.

Question 3 (5 Marks)

Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.

Question 4 (5 Marks)

Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.

Code of Conduct:

I K.DEEPIKA (192371042) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K.DEEPIKA

QUESTION 1 – RESPONSIVE WEB DESIGN

Introduction

Responsive Web Design is an approach to web development that allows a website to automatically adjust its layout and content according to the screen size and device being used. It enables a website to work properly on mobile phones, tablets, laptops, and desktop computers.

Importance of Responsive Web Design

Responsive web design is important because it provides a consistent and user-friendly experience across different devices. It improves readability and navigation, avoids unnecessary horizontal scrolling, reduces the need to create separate websites for different devices, and improves overall user satisfaction.

Key CSS Techniques for Responsiveness

The main CSS techniques used for responsive web design are flexible layouts, relative units, flexible images, media queries, Flexbox, and CSS Grid.

Flexible units such as percentages, em, rem, vw, and vh allow elements to adjust according to the available screen size. Flexible images prevent images from overflowing their containers. Media queries allow different styles to be applied depending on the screen size.

Flexbox and Grid

CSS Flexbox is useful for arranging elements in rows or columns and automatically adjusting their position according to the available space. For example, a navigation bar can display its items horizontally on a desktop and vertically on a mobile device.

CSS Grid is useful for creating layouts using rows and columns. It can be used to create responsive structures such as product grids, image galleries, and webpage sections.

Simple Example Layout

A webpage may contain three sections: Home, About, and Contact. On a desktop screen, these sections can be displayed side by side. On a mobile screen, they can be arranged vertically using media queries and Flexbox.

Desktop:

Home | About | Contact

Mobile:

Home
About
Contact

Improvements for Better User Experience

The website can be improved by using readable font sizes, touch-friendly buttons, flexible images, proper spacing, simple navigation, and mobile-friendly layouts. The website should also be tested on different screen sizes to ensure that all elements are displayed correctly.

Conclusion

Responsive web design allows websites to adapt to different screen sizes and devices. CSS techniques such as flexible layouts, media queries, Flexbox, and Grid help create websites that are responsive, accessible, and user-friendly.

QUESTION 2 – CLIENT-SIDE VALIDATION

Introduction

Client-side validation is the process of checking user input in the web browser before the form is submitted to the server. It ensures that the information entered by the user follows the required format.

Purpose of Client-Side Validation

The main purpose of client-side validation is to identify incorrect or missing information immediately. It provides quick feedback to users, reduces unnecessary server requests, improves form usability, and prevents common input mistakes.

For example, a registration form can check whether the name is entered, whether the email address has a valid format, and whether the phone number contains the required number of digits.

JavaScript Event Handling

JavaScript event handling is used to perform validation when a user interacts with a webpage. Common events include submit, click, input, and blur.

The submit event is commonly used for form validation. When the user clicks the submit button, JavaScript checks the entered information before allowing the form to be submitted.

Role of Regular Expressions

A regular expression, commonly called Regex, is a pattern used to identify whether text follows a particular format.

Regular expressions can be used to validate inputs such as email addresses, phone numbers, usernames, and passwords.

For example, an email should generally contain a username, an @ symbol, and a domain name. A phone number can be checked to ensure that it contains the required number of digits.

Example

Consider a registration form containing the following fields:

Name, Email, and Phone Number.

When the user clicks the Submit button, JavaScript checks whether all required fields are filled. It then checks whether the email follows a valid email format and whether the phone number contains ten digits. If the information is incorrect, an error message is displayed. If all information is valid, the form can be submitted.

Improvements for Usability and Security

Error messages should be clear and displayed near the incorrect field. Invalid fields can be highlighted to help users correct their mistakes easily. Required fields should be clearly indicated.

Client-side validation should not be considered a complete security mechanism because users can disable or bypass JavaScript. Therefore, all important validation must also be performed on the server side. User input should be validated and sanitized before being stored or processed.

Conclusion

Client-side validation improves the usability and efficiency of web forms by providing immediate feedback. JavaScript event handling performs the validation, while regular expressions help verify structured inputs such as email

addresses and phone numbers. Server-side validation should always be used for security.

QUESTION 3 – CSS BOX MODEL AND RESPONSIVE LAYOUT

Introduction

The CSS Box Model is a fundamental concept in CSS that describes how every HTML element is represented as a rectangular box. It determines the size, spacing, and appearance of elements on a webpage.

Components of the CSS Box Model

The CSS Box Model consists of four main components:

1.Content:

The content is the actual text, image, or other information contained inside an HTML element.

2.Padding:

Padding is the space between the content and the border of an element.

3.Border:

The border surrounds the content and padding of an element.

4.Margin:

Margin is the space outside the border. It creates distance between one element and other elements.

The order can be represented as:

Content → Padding → Border → Margin

CSS Positioning Techniques

CSS positioning determines how elements are placed on a webpage.

Static positioning is the default position of an element.

Relative positioning allows an element to be moved relative to its normal position.

Absolute positioning places an element relative to its nearest positioned parent.

Fixed positioning keeps an element fixed relative to the browser window, even when the page is scrolled.

Sticky positioning allows an element to behave normally until a particular scrolling position is reached, after which it remains fixed.

Causes of Layout Breaking

A webpage may break when the browser window is resized due to fixed-width elements, large images, excessive margins and padding, incorrect positioning, fixed font sizes, and the absence of responsive design techniques.

Improper use of absolute positioning can also cause elements to overlap or move outside their intended area.

Use of Media Queries

Media queries are CSS techniques used to apply different styles depending on the screen size or device.

For example, a webpage may display three sections horizontally on a desktop screen. When the screen becomes smaller, a media query can change the layout so that the sections are displayed vertically.

This helps the webpage adapt to mobile phones, tablets, and other screen sizes.

Best Practices for a Stable and Responsive Layout

The following practices should be followed:

1. Use flexible widths instead of fixed widths whenever possible.
2. Use Flexbox or CSS Grid for layouts.
3. Use media queries for different screen sizes.
4. Use flexible images that do not overflow their containers.
5. Avoid excessive use of absolute positioning.
6. Maintain consistent margins and padding.
7. Use appropriate font sizes.
8. Test the webpage on different devices and screen sizes.
9. Use the box-sizing property to control element dimensions properly.

10.Design with a mobile-friendly approach.

Conclusion

The CSS Box Model controls the size and spacing of webpage elements, while CSS positioning controls their placement. Layout problems caused by resizing can be reduced by using flexible layouts, media queries, Flexbox, Grid, and proper spacing techniques. These practices help create stable, responsive, and user-friendly webpages.

QUESTION 4 – DOCUMENT OBJECT MODEL (DOM)

Introduction

The **Document Object Model (DOM)** is a programming interface that represents an HTML webpage as a tree-like structure of objects. Each element, attribute, and piece of text in an HTML document is represented as a node in the DOM.

The DOM allows JavaScript to access and modify webpage elements without changing or reloading the entire webpage.

Role of DOM in Web Development

The DOM plays an important role in creating interactive and dynamic websites. It allows JavaScript to:

- Access HTML elements.
- Change text and content.
- Modify styles and attributes.
- Add or remove elements.
- Respond to user actions.
- Update webpage content dynamically.

For example, when a user clicks a button, JavaScript can use the DOM to change the text displayed on the page without reloading it.

DOM Structure

An HTML document can be represented as a tree:

Document

→**HTML**

→**Head**

→**Body**

→**Heading**

→**Paragraph**

→ **Button**

JavaScript can access these elements through the DOM and modify them when required.

JavaScript DOM Manipulation

JavaScript provides methods such as `getElementById()`, `querySelector()`, and `createElement()` to access and manipulate DOM elements.

For example, consider a webpage containing a button and a message. When the user clicks the button, JavaScript can change the message from “**Welcome**” to “**Welcome to our website!**” without reloading the page.

The basic process is:

User Action → JavaScript Event → DOM Element Selected → Content Updated → Page Changes

Example

Suppose a webpage displays:

Welcome

When the user clicks a button, the text can dynamically change to:

Welcome to our website!

This is achieved by selecting the required paragraph element through the DOM and changing its text content using JavaScript.

Advantages of Dynamic DOM Manipulation

- Updates content without refreshing the entire page.
- Provides faster interaction.
- Creates interactive web applications.
- Improves user experience.

- Reduces unnecessary page reloads.
- Allows webpages to respond immediately to user actions.

Improvements for Performance and User Experience

The performance of DOM-based applications can be improved by minimizing unnecessary DOM updates and modifying only the required elements. Frequently changing elements should be updated efficiently.

Other improvements include using event delegation for multiple elements, avoiding unnecessary JavaScript operations, loading scripts efficiently, and displaying loading indicators when dynamic data takes time to load.

For larger applications, asynchronous techniques such as **AJAX or Fetch API** can be used to retrieve data from the server without reloading the complete webpage.

Conclusion

The DOM provides a connection between HTML documents and JavaScript. It allows developers to access and modify webpage elements dynamically. By using DOM manipulation, JavaScript can update content without reloading the entire page, resulting in faster, more interactive, and user-friendly web applications.

QUESTION 1 – RESPONSIVE WEB DESIGN

Responsive Website

responsive-example.html

My Website

Home About Contact

Welcome to My Website

This is a simple example of a responsive website. Resize the browser window to see how the layout changes on different devices.

Desktop View
(Large Screen)

Home About Contact

Tablet View
(Medium Screen)

Home About Contact

Mobile View
(Small Screen)

Home
About
Contact

QUESTION 2 – CLIENT-SIDE VALIDATION

Form Validation

form-validation.html

Registration Form

Email:

Phone Number:

Please enter a valid email address.

QUESTION 3 – CSS BOX MODEL AND RESPONSIVE LAYOUT

Box Model Example

box-model.html

CSS Box Model

This box demonstrates the Box Model: Content, Padding, Border and Margin.



Responsive Layout

Resize the browser window to see how the layout changes.

Desktop View
(Wide Screen)

Box 1 Box 2 Box 3

Tablet View
(Medium Screen)

Box 1
Box 2
Box 3

Mobile View
(Small Screen)

Box 1
Box 2
Box 3